

MODUL III

ARRAY

3.1 TUJUAN PRAKTIKUM

Tujuan Praktikum pada Modul III *Array*:

1. Mahasiswa mampu memahami pengertian dan penggunaan *array*.
2. Mahasiswa mampu membuat program dengan menggunakan *array*.
3. Mahasiswa mampu memahami cara memanipulasi *string*.
4. Mahasiswa mampu membuat program yang mampu memanipulasi *string*.

3.2 DASAR TEORI

3.2.1 *Array*

Array (larik) merupakan struktur data yang digunakan untuk menyimpan sekumpulan data dengan tipe yang sama dalam satu variabel secara terorganisir [1]. Struktur data ini memudahkan proses pengolahan data karena banyak nilai dapat disimpan dan diakses menggunakan satu nama variabel dengan bantuan indeks [2]. Dalam Bahasa pemrograman C++, indeks *array* dimulai dari 0, sehingga elemen pertama berada pada indeks ke-0 [3].

Array disimpan dalam lokasi memori yang berurutan (*contiguous memory*), sehingga setiap elemen dapat diakses secara langsung menggunakan indeks tertentu [1]. Salah satu karakteristik *array* adalah seluruh elemen di dalamnya harus memiliki tipe data yang sama, seperti *integer*, *float*, maupun *character* [4]. Selain itu, ukuran *array* biasanya ditentukan saat deklarasi dan bersifat tetap selama program dijalankan [2]. Bentuk umum deklarasi *array* pada C++ adalah sebagai berikut:

```
tipe_data nama_array[indeks];
```

Berdasarkan bentuk deklarasi *array* tersebut, terdapat beberapa bagian penting yang memiliki fungsi masing-masing, yaitu:

1. “tipe_data” digunakan untuk menentukan jenis data yang akan disimpan dalam *array*, seperti *int*, *float*, atau *char*.
2. “nama_array” digunakan sebagai identitas atau nama variabel *array* yang akan digunakan dalam program.
3. “[indeks]” digunakan untuk menentukan kapasitas maksimum elemen yang dapat disimpan dalam *array*.

Elemen-elemen dalam *array* dapat diakses menggunakan indeks *array*, di mana indeks pertama selalu dimulai dari 0 [2].

Contoh pengaksesan *array* sebagai berikut:

```
char nama [] = "Alpro";  
cout << nama [1];
```

Kode tersebut akan menampilkan karakter kedua dari *array* “nama”, yaitu “1”. Pengaksesan *array* juga dapat dilakukan menggunakan perulangan (*looping*) untuk membaca seluruh elemen *array* secara berurutan [1]. *Array* memiliki beberapa jenis berdasarkan jumlah dimensinya [1], yaitu sebagai berikut:

1. *Array* Satu Dimensi

Array satu dimensi adalah *array* yang hanya memiliki satu indeks dan digunakan untuk menyimpan data secara linear atau satu baris [2]. Pengaksesan elemen pada *array* ini dilakukan menggunakan satu indeks, dimulai dari indeks ke-0 [3]. Bentuk umum *array* satu dimensi:

```
tipe_data nama_array [indeks];
```

Contoh *array* satu dimensi:

```
int angka [5] = {10, 20, 30, 40, 50};
```

Pada contoh di atas, *array* “angka” memiliki 5 elemen bertipe *integer*.

2. *Array* Dua Dimensi

Array dua dimensi adalah *array* yang memiliki dua indeks dan biasanya digunakan untuk menyimpan data dalam bentuk tabel atau matriks [1]. Indeks pertama menunjukkan baris, sedangkan indeks kedua menunjukkan kolom [4]. Bentuk umum *array* dua dimensi:

```
tipe_data nama_array [indeks1] [indeks2];
```

Contoh *array* dua dimensi:

```
int matriks [2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};
```

Array di atas memiliki 2 baris dan 3 kolom.

3. *Array* Multidimensi

Array multidimensi merupakan pengembangan dari *array* dua dimensi yang memiliki lebih dari dua indeks [3]. Jenis *array* ini digunakan untuk menyimpan data yang lebih kompleks. Bentuk umum *array* multidimensi:

```
tipe_data nama_array [indeks1] [indeks2] [indeksn];
```

Contoh *array* multidimensi:

```
int data [2][2][2];
```

3.2.2 String

String dalam bahasa pemrograman C++ merupakan Kumpulan karakter yang disusun secara berurutan dan disimpan dalam bentuk *array* karakter [1]. Setiap karakter dalam string memiliki indeks tertentu sehingga dapat diakses seperti *array* biasa [2].

String selalu diakhiri dengan karakter khusus yaitu *null terminator* (`'\0'`) yang berfungsi sebagai penanda akhir *string* [3]. Sebagai contoh, *string* "Alpro" sebenarnya disimpan seperti sebagai berikut:

```
{ 'A', 'l', 'p', 'r', 'o', '\0' }
```

Karena *string* disimpan sebagai *array* karakter, maka setiap karakter dapat diakses menggunakan indeks [1]. Contoh pengaksesan *string*:

```
char nama [] = "Alpro";
cout << nama [1];
```

Kode tersebut digunakan untuk mengakses karakter pada *string* menggunakan indeks *array*. Pada *string* "Alpro", "nama [1]" akan mengakses karakter kedua, yaitu huruf "l", kemudian ditampilkan dengan "cout".

String dapat dimanipulasi untuk berbagai kebutuhan, seperti menghitung panjang *string*, menggabungkan *string*, maupun mengubah karakter tertentu [1]. Manipulasi *string* dapat dilakukan menggunakan perulangan ataupun fungsi tertentu [4].

Contoh manipulasi *string* untuk mengubah huruf vokal menjadi karakter "*" adalah sebagai berikut:

```
#include <iostream>
using namespace std;

int main () {
    char kalimat [100];
    int i = 0;

    cout << "Masukkan kalimat: ";
    cin.getline(kalimat, 100);

    while (kalimat[i] != '\0') {
        if (kalimat[i] == 'a' || kalimat[i] == 'i' ||
            kalimat[i] == 'u' || kalimat[i] == 'e' ||
            kalimat[i] == 'o') {
            kalimat[i] = '*';
        }
        i++;
    }

    cout << "Hasil manipulasi string: " << kalimat;

    return 0;
}
```

3.3 PERMASALAHAN

Tahun 2167 Stasiun luar angkasa S.K.E.L.D.I. mengapung diam di orbit Saturnus. Tiba-tiba, alarm berbunyi keras, sistem utama kapal mati satu per satu. Seorang impostor telah menyabotase kapal dari dalam. Kamu adalah satu dari sedikit crew yang masih sadar. Tidak ada waktu untuk rapat. Tidak ada waktu untuk vote. Satu-satunya cara menyelamatkan kapal adalah menyelesaikan semua task sebelum impostor menyelesaikan misinya. Komputer darurat menyala redup dan menampilkan 3 task yang harus diselesaikan. Kamu menarik napas dalam, dan mulai mengetik.

1. *Decode Signal*: Pesan Terenkripsi Dari Bumi.

Sistem komunikasi di *Communication Room* terenkripsi. Sistem enkripsi kapal menggunakan dua lapis perlindungan: penyusunan ulang karakter ke dalam matriks dengan 4 kolom, lalu pergeseran nilai ASCII berdasarkan sebuah *key*. Tanpa dekripsi yang tepat, koordinat evakuasi tidak akan terbaca dan tidak ada yang bisa mengarahkan petugas penyelamat ke lokasi yang benar.

Kamu harus membangun program *Columnar Transposition* dan *Caesar Cipher* yang dapat melakukan enkripsi pesan dengan validasi *input* yang ketat, menampilkan matriks enkripsi, dan hasil cipher. Program juga harus mampu melakukan dekripsi untuk memverifikasi bahwa pesan dapat dipulihkan dengan benar.

```
[TASK 1/3] DECODE SIGNAL - Communication Room
>> Kirim Sinyal terenkripsi dan dekripsi balasan dari Bumi!.

[ ENKRIPSI ]

Masukkan key           : 3
Masukkan teks          : Si Hitam Impostornya!

Matriks enkripsi:

S i  H
i t a m
I m p
o s t o
r n y a
! - - -

Enkripsi sebelum ditambah key : Si or!itIsn_ amty_Hmpoa_
Enkripsi setelah ditambah key  : Vl#ru$lwLvqb#dpw|bKpsrdb
```

Gambar 3.1 Tampilan Program Enkripsi Pesan dengan Matriks 4 Kolom.

```
[ DEKRIPSI ]

Masukkan teks terenkripsi : P#BdHbqdbjnb
Dekripsi setelah dikurang key : M ?aE_na_gk_

Matriks Dekripsi:

M a n g
E a k
? - - -

Hasil Dekripsi final      : Mang Eak?
>> [TASK 1/3] COMPLETE - Signal channel restored.
>> Tekan ENTER untuk melanjutkan ke [ TASK 2 ]!.
```

Gambar 3.2 Tampilan Program Dekripsi Pesan dan Verifikasi Integritas.

2. *Repair Text Console*: Pemulihan Modul Teks Kapten.

Konsol utama di *Captain's Bridge* rusak akibat interferensi kosmik dari badai partikel Saturnus. Data di dalamnya kacau dan tidak terbaca. Kapten membutuhkan laporan situasi dengan analisis mendalam tentang data teks yang masuk, termasuk verifikasi *palindrome*, ekstraksi karakter, dan manipulasi *string* untuk memastikan integritas data.

Program ini harus memproses tiga kalimat secara bersamaan dalam satu siklus. Untuk setiap kalimat, program akan menghapus spasi, menampilkan jumlah huruf, menampilkan karakter asli dan *inverse*-nya, melakukan *swap* pada karakter pertama dan terakhir, serta melakukan pengecekan *palindrome* manual per karakter. Program berjalan dalam *loop* yang memungkinkan *user* untuk mengulang operasi berkali-kali.

```
[TASK 2/3] REPAIR TEXT CONSOLE - Captain's Bridge
>> Konsol teks mengalami interferensi kosmik.
>> Verifikasi integritas data teks sebelum terlambat!

[ Verifikasi Teks ]

Masukkan Kalimat ke-1 : Abib na atu
-----
Hasil untuk Kalimat ke-1:
Jumlah Huruf          : 9
1. Asli               : A, b, i, b, n, a, a, t, u
2. Inverse             : u, t, a, a, n, b, i, b, A
3. Swap Awal-Akhir    : ubibnaatA
4. Palindrome?        : NO
```

Gambar 3.3 Tampilan Pemrosesan Kalimat *Loop* Pertama.

```
Masukkan Kalimat ke-2 : bibib
-----
Hasil untuk Kalimat ke-2:
Jumlah Huruf          : 5
1. Asli               : b, i, b, i, b
2. Inverse             : b, i, b, i, b
3. Swap Awal-Akhir    : bibib
4. Palindrome?        : YES

Masukkan Kalimat ke-3 : kasur rusak
-----
Hasil untuk Kalimat ke-3:
Jumlah Huruf          : 10
1. Asli               : k, a, s, u, r, r, u, s, a, k
2. Inverse             : k, a, s, u, r, r, u, s, a, k
3. Swap Awal-Akhir    : kasur rusak
4. Palindrome?        : YES

=====
Verifikasi Teks Lain? (y/n): n

>> [TASK 2/3] COMPLETE - Text console back online.
>> Tekan ENTER untuk melanjutkan ke [ TASK 3 ]!.
```

Gambar 3.4 Tampilan *Loop* Kedua dan Ketiga serta Verifikasi Teks.

3. *Align Navigation Matrix*: Rekonstruksi Sistem Navigasi.

Sistem navigasi di *Engine Core* sepenuhnya mati. Matriks koordinat yang mengontrol manuver kapal rusak total. Tanpa matriks yang benar, kapal tidak bisa bermanuver dan akan terseret gravitasi Saturnus dalam hitungan jam. Satu-satunya cara adalah merekonstruksi matriks dari data mentah dan menjalankan tiga operasi transformasi secara berurutan. Tidak boleh ada satu langkah pun yang salah.

Program harus meminta ukuran matriks antara 3 sampai 5, kemudian membuat dua matriks, yaitu matriks ganjil dan matriks genap. Kemudian menjalankan tiga operasi transformasi secara berurutan dengan urutan *Fusion Matrix* atau menjumlahkan kedua matriks, *Matrix Axis Swap* atau men-*transpose* hasil *fusion*, *Snake Traverse* atau membaca matriks hasil *transpose* dengan pola *zig-zag*. Program menampilkan hasil dari setiap operasi transformasi.

```
[TASK 3/3] ALIGN NAVIGATION MATRIX - Engine Core
>> Sistem navigasi mati total akibat sabotase.
>> Rekonstruksi matriks koordinat sekarang!

[ MATRIX NAVIGATION ]

Masukkan ukuran Navigation Matrix (max 5): 5
-----
Odd Coordinate Matrix (Ganjil):
01 03 05 07 09
11 13 15 17 19
21 23 25 27 29
31 33 35 37 39
41 43 45 47 49

Even Coordinate Matrix (Genap):
02 04 06 08 10
12 14 16 18 20
22 24 26 28 30
32 34 36 38 40
42 44 46 48 50
-----

Matriks koordinat telah dipulihkan
Lakukanlah proses-proses berikut ini untuk membuat matriks koordinat lebih akurat!
1. Coordinate Matrix Integration
2. Matrix Transpose
3. Snake Traverse
```

Gambar 3.5 Tampilan Pembuatan Matriks Ganjil dan Genap.

```
COORDINATE MATRIX INTEGRATION:
-----
03 07 11 15 19
23 27 31 35 39
43 47 51 55 59
63 67 71 75 79
83 87 91 95 99

MATRIX TRANSPOSE:
-----
03 23 43 63 83
07 27 47 67 87
11 31 51 71 91
15 35 55 75 95
19 39 59 79 99
```

Gambar 3.6 Tampilan Operasi *Fusion Matrix* dan *Transpose*.

```
SNAKE TRAVERSE:
-----
03 23 43 63 83
87 67 47 27 07
11 31 51 71 91
95 75 55 35 15
19 39 59 79 99
-----

>> [TASK 3/3] COMPLETE - Navigation matrix aligned.
=====
ALL TASKS COMPLETE - SHIP SYSTEMS RESTORED
=====
```

Gambar 3.7 Tampilan *Snake Traverse* dan Hasil Akhir Navigasi.

3.4 HASIL PERCOBAAN

1. Algoritma

- a. Mulai.
- b. Deklarasi variabel MAX, key, teks, enkripsi, panjang, panjangEnkripsi, baris, barisDekripsi, indeks, indeks2, i, j, pilih, kalimat1, kalimat2, kalimat3, panjangKalimat1, panjangKalimat2, panjangKalimat3, jumlah1, jumlah2, jumlah3, awal, akhir, kiri, kanan, palindrome, simpan, n, nilaiGanjil, nilaiGenap, ganjil, genap, gabungGG, dan transpose.
- c. Tampilkan *header* “[TASK 1/3] DECODE SIGNAL - Communication Room”.
- d. *Input key*, jika $key \leq 0$ maka ulangi *input*.
- e. *Input teks*, jika kosong maka ulangi *input*.
- f. Hitung panjang teks dan jumlah baris matriks.
- g. Isi matriks enkripsi secara horizontal dan isi sisa elemen dengan ‘_’.
- h. Tampilkan matriks enkripsi, hasil enkripsi sebelum *key*, dan hasil enkripsi sesudah *key*.
- i. *Input teks* terenkripsi, jika kosong maka ulangi *input*.
- j. Kurangi setiap karakter terenkripsi dengan *key* lalu isi matriks dekripsi per kolom.
- k. Tampilkan matriks dekripsi dan hasil dekripsi final tanpa karakter ‘_’.
- l. Tampilkan pesan “[TASK 1/3] COMPLETE”.
- m. Tampilkan *header* “[TASK 2/3] REPAIR TEXT CONSOLE - Captain’s Bridge”.
- n. *Input* 3 kalimat, hitung jumlah huruf selain spasi, tampilkan karakter asli, *inverse*, *swap* awal-akhir, dan hasil *palindrome* YES atau NO.
- o. *Input* pilihan verifikasi ulang (y/n), jika pilih = y maka ulangi proses *input* kalimat.
- p. Tampilkan pesan “[TASK 2/3] COMPLETE”.
- q. Tampilkan *header* “[TASK 3/3] ALIGN NAVIGATION MATRIX - Engine Core”.
- r. *Input* ukuran matriks n, jika $n < 3$ atau $n > 5$ maka ulangi *input*.
- s. Isi dan tampilkan matriks ganjil serta matriks genap ukuran $n \times n$.
- t. Jumlahkan kedua matriks ke matriks gabungGG lalu tampilkan hasil *integration*.
- u. *Transpose* matriks gabungGG lalu tampilkan hasil *transpose*.
- v. Tampilkan *snake traverse* dengan baris genap dibaca kiri ke kanan dan baris ganjil dibaca kanan ke kiri.
- w. Tampilkan pesan “[TASK 3/3] COMPLETE - Navigation matrix aligned”.
- x. Tampilkan pesan “ALL TASK COMPLETE - SHIP SYSTEM RESTORED”.
- y. Selesai.

2. Pseudocode

```

Step 1:  MULAI
Step 2:  DEKLARASI MAX, key, teks, enkripsi, panjang, baris, indeks,
        pilih, kalimat1, kalimat2, kalimat3, jumlah, awal, akhir,
        kiri, kanan, palindrome, simpan, n, ganjil, genap, gabungGG,
        transpose, i, j
Step 3:  PRINT "==== [TASK 1/3] DECODE SIGNAL ====="
Step 4:  WHILE (TRUE)
Step 5:      INPUT key
Step 6:      IF (key > 0)
Step 7:          BREAK
Step 8:      ELSE
Step 9:          PRINT "Key tidak valid!"
Step 10:     END IF
Step 11:  END WHILE
Step 12:  WHILE (TRUE)
Step 13:      INPUT teks
Step 14:      IF (teks tidak kosong)
Step 15:          BREAK
Step 16:      ELSE
Step 17:          PRINT "Input tidak boleh kosong!"
Step 18:      END IF
Step 19:  END WHILE
Step 20:  HITUNG panjang teks
Step 21:  SET baris = (panjang + 3) / 4
Step 22:  ISI matriks enkripsi secara horizontal
Step 23:  JIKA kosong isi '_'
Step 24:  PRINT matriks enkripsi
Step 25:  PRINT hasil enkripsi sebelum key
Step 26:  PRINT hasil enkripsi sesudah key
Step 27:  PRINT "==== [DESKRIPSI] ====="
Step 28:  WHILE (TRUE)
Step 29:      INPUT enkripsi
Step 30:      IF (enkripsi tidak kosong)
Step 31:          BREAK
Step 32:      ELSE
Step 33:          PRINT "Input tidak boleh kosong!"
Step 34:      END IF
Step 35:  END WHILE
Step 36:  HITUNG panjang enkripsi
Step 37:  FOR i = 0 sampai panjangEnkripsi
Step 38:      enkripsi[i] = enkripsi[i] - key
Step 39:  END FOR
Step 40:  ISI matriks dekripsi per kolom
Step 41:  PRINT matriks dekripsi
Step 42:  PRINT hasil dekripsi final
Step 43:  PRINT "[TASK 1/3] COMPLETE"
Step 44:  PRINT "==== [TASK 2/3] REPAIR TEXT CONSOLE ====="
Step 45:  WHILE (TRUE)
Step 46:      INPUT kalimat1
Step 47:      HITUNG jumlah huruf
Step 48:      PRINT karakter asli
Step 49:      PRINT inverse
Step 50:      PRINT swap awal-akhir

```

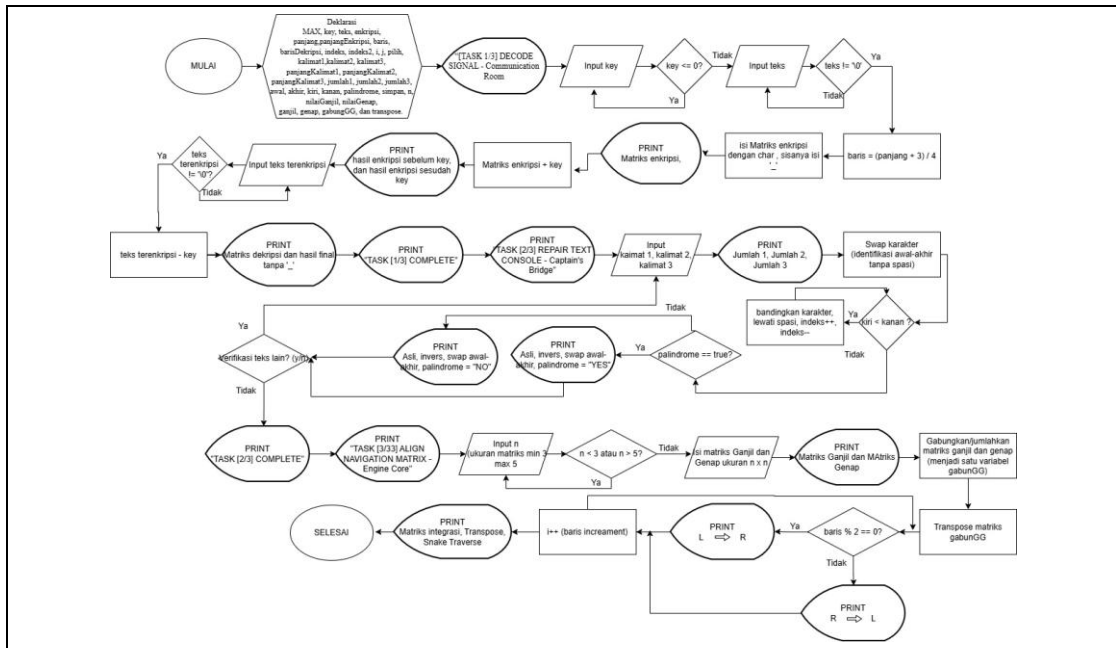


```

Step 51:    PRINT hasil palindrome
Step 52:    INPUT kalimat2
Step 53:    HITUNG jumlah huruf
Step 54:    PRINT karakter asli
Step 55:    PRINT inverse
Step 56:    PRINT swap awal-akhir
Step 57:    PRINT hasil palindrome
Step 58:    INPUT kalimat3
Step 59:    HITUNG jumlah huruf
Step 60:    PRINT karakter asli
Step 61:    PRINT inverse
Step 62:    PRINT swap awal-akhir
Step 63:    PRINT hasil palindrome
Step 64:    INPUT pilih
Step 65:    IF (pilih = 'n')
Step 66:        BREAK
Step 67:    END IF
Step 68:    END WHILE
Step 69:    PRINT "[TASK 2/3] COMPLETE"
Step 70:    PRINT "==== [TASK 3/3] ALIGN NAVIGATION MATRIX ====="
Step 71:    WHILE (TRUE)
Step 72:        INPUT n
Step 73:        IF (n >= 3 AND n <= 5)
Step 74:            BREAK
Step 75:        ELSE
Step 76:            PRINT "Ukuran matriks tidak valid!"
Step 77:        END IF
Step 78:    END WHILE
Step 79:    ISI matriks ganjil
Step 80:    PRINT matriks ganjil
Step 81:    ISI matriks genap
Step 82:    PRINT matriks genap
Step 83:    JUMLAHKAN matriks ganjil dan genap
Step 84:    PRINT hasil integration
Step 85:    TRANSPOSE matriks gabungGG
Step 86:    PRINT hasil transpose
Step 87:    PRINT snake traverse
Step 88:    PRINT "[TASK 3/3] COMPLETE"
Step 89:    PRINT "ALL TASK COMPLETE - SHIP SYSTEM RESTORED"
Step 90:    SELESAI

```

3. Flowchart



Gambar 3.8 Flowchart Program Stasiun luar angkasa S.K.E.L.D.I.

4. Source Code

```
#include <iostream>
using namespace std;
int main(){
    const int MAX = 100;
    int key;
    char teks [MAX];
    cout << "[ TASK 1/3 ] DECODE SIGNAL - Communication Room" << endl
    << endl;
    cout << ">> Kirim sinyal terenkripsi dan deskripsi balasan dari
    Bumi!." << endl << endl;
    cout << "[ ENKRIPSI ]" << endl << endl;
    do{
        cout << "Masukkan key \t\t: "; cin >> key;
        if (cin.fail() || key <= 0){
            cin.clear();
            cin.ignore (MAX, '\n');
            cout << "[!] Key harus bilangan bulat positif. Coba lagi" << endl
            << endl;
        }
    }while (key <= 0);
    cin.ignore (MAX, '\n');
    do{
        cout << "Masukkan teks \t\t: ";
        cin.getline (teks, MAX);
        cout << endl;
        if (teks [0] == '\0'){
```

```

        cout << "[!] Input tidak boleh kosong. Coba lagi" << endl <<
endl;
    }
    while(teks [0] == '\0');
    //Matriks enkripsi
    int panjang = 0;
    while (teks [panjang] != '\0'){
        panjang++;
    }
    int baris = (panjang + 3) / 4; //buat itung jumlah barisnya
    char matriks [baris][4];
    int indeks = 0;
    for (int i = 0 ; i < baris ; i++){
        for (int j = 0 ; j < 4 ; j++){
            if (indeks < panjang){
                matriks [i][j] = teks [indeks];
            }
            else {
                matriks [i][j] = '_';
            }
            indeks++;
        }
    }
    cout << "Matriks Enkripsi: " << endl << endl;
    for (int i = 0 ; i < baris ; i++){
        for (int j = 0 ; j < 4 ; j++){
            cout << matriks [i][j] << " ";
        }
        cout << endl << endl;
    }
    cout << "Enkripsi sebelum ditambah key \t: ";
    for (int j = 0; j < 4; j++) {
        for (int i = 0; i < baris; i++) {
            cout << matriks[i][j];
        }
    }
    cout << endl;
    cout << "Enkripsi setelah ditambah key \t: ";
    for (int j = 0; j < 4; j++) {
        for (int i = 0; i < baris; i++) {
            cout << (char)(matriks[i][j] + key);
        }
    }
    cout << endl << endl;
    cout << "[ DESKRIPSI ]" << endl << endl;
    char enkripsi[MAX];
    // cin.ignore(MAX, '\n');
    do {

```

```

        cout << "Masukkan teks terenkripsi \t: ";
        cin.getline(enkripsi, MAX);
        if (enkripsi[0] == '\0') {
            cout << "[!] Input tidak boleh kosong. Coba lagi" << endl;
        }
    } while (enkripsi[0] == '\0');
    int panjangEnkripsi = 0;
    while (enkripsi[panjangEnkripsi] != '\0') {
        panjangEnkripsi++;
    }
    for (int i = 0; i < panjangEnkripsi; i++) {
        enkripsi[i] = enkripsi[i] - key;
    }
    cout << "Dekripsi setelah dikurang key \t: ";
    for (int i = 0; i < panjangEnkripsi; i++) {
        cout << enkripsi[i];
    }
    cout << endl << endl;
    int barisDekripsi = (panjangEnkripsi + 3) / 4;
    char matriksDekripsi[MAX][4];
    int indeks2 = 0;
    for (int j = 0; j < 4; j++) {
        for (int i = 0; i < barisDekripsi; i++) {
            if (indeks2 < panjangEnkripsi) {
                matriksDekripsi[i][j] = enkripsi[indeks2];
            } else {
                matriksDekripsi[i][j] = '_';
            }
            indeks2++;
        }
    }
    cout << "Matriks Dekripsi:" << endl << endl;
    for (int i = 0; i < barisDekripsi; i++) {
        for (int j = 0; j < 4; j++) {
            cout << matriksDekripsi[i][j] << " ";
        }
        cout << endl;
    }
    cout << endl << "Hasil Dekripsi Final \t: ";
    for (int i = 0; i < barisDekripsi; i++) {
        for (int j = 0; j < 4; j++) {
            if (matriksDekripsi[i][j] != '_') {
                cout << matriksDekripsi[i][j];
            }
        }
    }
    cout << endl;
    cout << ">> [TASK 1/3] COMPLETE - Signal channel restored." << endl;

```

```

cout << "Tekan ENTER untuk melanjutkan ke [ TASK 2 ] !" << endl;
system ("pause");
system ("cls");
cout << endl;
cout << "[TASK 2/3] REPAIR TEXT CONSOLE - Captain's Bridge" << endl;
cout << ">> Konsol teks mengalami interfensi kosmik." << endl;
cout << ">> Verifikasi integritas data teks sebelum terlambat!" <<
endl << endl;
char pilih;
do {
    cout << "[VERIFIKASI TEKS]" << endl << endl;
    char kalimat1[MAX];
    cin.ignore(MAX, '\n');
    cout << "Masukkan Kalimat ke-1 \t: ";
    cin.getline(kalimat1, MAX);
    int panjangKalimat1 = 0;
    int jumlah1 = 0;
    while (kalimat1[panjangKalimat1] != '\0') {
        if (kalimat1[panjangKalimat1] != ' ') {
            jumlah1++;
        }
        panjangKalimat1++;
    }
    cout << "-----" << endl;
    cout << "Hasil untuk Kalimat ke-1:" << endl;
    cout << "Jumlah Huruf \t\t: " << jumlah1 << endl;
    cout << "1. Asli \t\t: ";
    for (int i = 0; i < panjangKalimat1; i++) {
        if (kalimat1[i] != ' ') {
            cout << kalimat1[i];
            if (i < panjangKalimat1 - 1) {
                cout << ", ";
            }
        }
    }
    cout << endl;
    cout << "2. Inverse \t\t: ";
    for (int i = panjangKalimat1 - 1; i >= 0; i--) {
        if (kalimat1[i] != ' ') {
            cout << kalimat1[i];
            if (i > 0) {
                cout << ", ";
            }
        }
    }
    cout << endl;
    int awal = 0;
    int akhir = panjangKalimat1 - 1;

```

```

while (kalimat1[awal] == ' ') awal++;
while (kalimat1[akhir] == ' ') akhir--;
char simpan = kalimat1[awal];
kalimat1[awal] = kalimat1[akhir];
kalimat1[akhir] = simpan;
cout << "3. Swap Awal-Akhir \t: ";
for (int i = 0; i < panjangKalimat1; i++) {
    if (kalimat1[i] != ' ') {
        cout << kalimat1[i];
    }
}
cout << endl;
int kiri = 0;
int kanan = panjangKalimat1 - 1;
bool palindrome = (jumlah1 != 0);
while (kiri < kanan) {
    while (kalimat1[kiri] == ' ') kiri++;
    while (kalimat1[kanan] == ' ') kanan--;
    if (kalimat1[kiri] != kalimat1[kanan]){
        palindrome = false;
        break;
    }
    kiri++;
    kanan--;
}
cout << "4. Palindrome? \t\t: ";
if (palindrome) {
    cout << "YES" << endl;
} else {
    cout << "NO" << endl;
}
cout << endl;
char kalimat2[MAX];
cout << "Masukkan Kalimat ke-2 \t: ";
cin.getline(kalimat2, MAX);
int panjangKalimat2 = 0;
int jumlah2 = 0;
while (kalimat2[panjangKalimat2] != '\0') {
    if (kalimat2[panjangKalimat2] != ' ') {
        jumlah2++;
    }
    panjangKalimat2++;
}
cout << "-----" << endl;
cout << "Hasil untuk Kalimat ke-2:" << endl;
cout << "Jumlah Huruf \t\t: " << jumlah2 << endl;
cout << "1. Asli \t\t: ";
for (int i = 0; i < panjangKalimat2; i++) {

```

```

        if (kalimat2[i] != ' ') {
            cout << kalimat2[i];
            if (i < panjangKalimat2 - 1) cout << ", ";
        }
    }
    cout << endl;
    cout << "2. Inverse \t\t: ";
    for (int i = panjangKalimat2 - 1; i >= 0; i--) {
        if (kalimat2[i] != ' ') {
            cout << kalimat2[i];
            if (i > 0) cout << ", ";
        }
    }
    cout << endl;
    awal = 0;
    akhir = panjangKalimat2 - 1;
    while (kalimat2[awal] == ' ') awal++;
    while (kalimat2[akhir] == ' ') akhir--;
    simpan = kalimat2[awal];
    kalimat2[awal] = kalimat2[akhir];
    kalimat2[akhir] = simpan;
    cout << "3. Swap Awal-Akhir \t: ";
    for (int i = 0; i < panjangKalimat2; i++) {
        if (kalimat2[i] != ' ') cout << kalimat2[i];
    }
    cout << endl;
    kiri = 0;
    kanan = panjangKalimat2 - 1;
    palindrome = (jumlah2 != 0);
    while (kiri < kanan) {
        while (kalimat2[kiri] == ' ') kiri++;
        while (kalimat2[kanan] == ' ') kanan--;
        if (kalimat2[kiri] != kalimat2[kanan]) {
            palindrome = false;
            break;
        }
        kiri++;
        kanan--;
    }
    cout << "4. Palindrome? \t\t: ";
    if (palindrome)
        cout << "YES" << endl;
    else
        cout << "NO" << endl;
    cout << endl;
    char kalimat3[MAX];
    cout << "Masukkan Kalimat ke-3 \t: ";
    cin.getline(kalimat3, MAX);

```

```

int panjangKalimat3 = 0;
int jumlah3 = 0;
while (kalimat3[panjangKalimat3] != '\0') {
    if (kalimat3[panjangKalimat3] != ' ') {
        jumlah3++;
    }
    panjangKalimat3++;
}
cout << "-----" << endl;
cout << "Hasil untuk Kalimat ke-3:" << endl;
cout << "Jumlah Huruf \t\t: " << jumlah3 << endl;
cout << "1. Asli \t\t: ";
for (int i = 0; i < panjangKalimat3; i++) {
    if (kalimat3[i] != ' ') {
        cout << kalimat3[i];
        if (i < panjangKalimat3 - 1) cout << ", ";
    }
}
cout << endl;
cout << "2. Inverse \t\t: ";
for (int i = panjangKalimat3 - 1; i >= 0; i--) {
    if (kalimat3[i] != ' ') {
        cout << kalimat3[i];
        if (i > 0) cout << ", ";
    }
}
cout << endl;
awal = 0;
akhir = panjangKalimat3 - 1;
while (kalimat3[awal] == ' ') awal++;
while (kalimat3[akhir] == ' ') akhir--;
simpan = kalimat3[awal];
kalimat3[awal] = kalimat3[akhir];
kalimat3[akhir] = simpan;
cout << "3. Swap Awal-Akhir \t: ";
for (int i = 0; i < panjangKalimat3; i++) {
    if (kalimat3[i] != ' ') cout << kalimat3[i];
}
cout << endl;
kiri = 0;
kanan = panjangKalimat3 - 1;
palindrome = (jumlah3 != 0);
while (kiri < kanan) {
    while (kalimat3[kiri] == ' ') kiri++;
    while (kalimat3[kanan] == ' ') kanan--;
    if (kalimat3[kiri] != kalimat3[kanan]) {
        palindrome = false;
        break;
    }
}

```



```

    }
    kiri++;
    kanan--;
}
cout << "4. Palindrome? \t\t: ";
if (palindrome) cout << "YES" << endl;
else cout << "NO" << endl;
cout << endl;
cout << "===== "
<< endl;
do {
    cout << "Verifikasi Teks Lain? (y/n): ";
    cin >> pilih;
    if (cin.fail() || (pilih != 'y' && pilih != 'Y' && pilih != 'n'
&& pilih != 'N')) {
        cin.clear();
        cin.ignore(MAX, '\n');
        cout << "[!] Input tidak valid. Masukkan y atau n." << endl;
    }
} while (cin.fail() || (pilih != 'y' && pilih != 'Y' && pilih != 'n'
&& pilih != 'N'));
} while (pilih == 'y' || pilih == 'Y');
cout << endl << ">> [TASK 2/3] COMPLETE - Text console back online."
<< endl;
cout << ">> Tekan ENTER untuk melanjutkan ke [ TASK 3 ]!." << endl;
system("pause");
system("cls");
cout << endl;
cout << "[ TASK 3/3 ] ALIGN NAVIGATION MATRIX - Engine Core" << endl;
cout << ">> Sistem navigasi mati total akibat sabotase." << endl;
cout << ">> Rekonstruksi matriks koordinat sekarang!" << endl <<
endl;
cout << "[ MATRIX NAVIGATION ]" << endl << endl;
int n;
do {
    cout << "Masukkan ukuran Navigation Matrix (min 3, max 5): ";
    cin >> n;
    if (cin.fail() || n < 3 || n > 5) {
        cin.clear();
        cin.ignore(MAX, '\n');
        cout << "[!] Input tidak valid. Masukkan ukuran antara 3
sampai 5." << endl << endl;
    }
} while (cin.fail() || n < 3 || n > 5);
int ganjil[5][5];
int genap[5][5];
cout << "Odd Coordinate Matrix (Ganjil):" << endl;
int nilaiGanjil = 1;

```

```

for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ganjil[i][j] = nilaiGanjil;
        if (ganjil[i][j] < 10) cout << " 0" << ganjil[i][j];
        else cout << " " << ganjil[i][j];
        nilaiGanjil += 2;
    }
    cout << endl;
}
cout << endl;
cout << "Even Coordinate Matrix (Genap):" << endl;
int nilaiGenap = 2;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        genap[i][j] = nilaiGenap;
        if (genap[i][j] < 10) cout << " 0" << genap[i][j];
        else cout << " " << genap[i][j];
        nilaiGenap += 2;
    }
    cout << endl;
}
cout << "-----"
" << endl;
cout << endl << "Matriks koordinat telah dipulihkan" << endl;
cout << "Lakukanlah proses-proses berikut ini untuk membuat matriks
koordinat lebih akurat!" << endl;
cout << "1. Coordinate Matrix Integration" << endl;
cout << "2. Matrix Transpose" << endl;
cout << "3. Snake Traverse" << endl << endl;
cout << "Tekan ENTER untuk melanjutkan..." << endl;
system("pause");
system("cls");
int gabungGG[5][5];
cout << "COORDINATE MATRIX INTEGRATION:" << endl;
cout << "-----"
" << endl;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        gabungGG[i][j] = ganjil[i][j] + genap[i][j];
        if (gabungGG[i][j] < 10) cout << " 0" << gabungGG[i][j];
        else cout << " " << gabungGG[i][j];
    }
    cout << endl;
}
cout << endl;
cout << "MATRIX TRANSPOSE:" << endl;
cout << "-----"
" << endl;

```

```

int transpose[5][5];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        transpose[i][j] = gabungGG[j][i];
        if (transpose[i][j] < 10) cout << " 0" << transpose[i][j];
        else cout << " " << transpose[i][j];
    }
    cout << endl;
}
cout << endl;
cout << "SNAKE TRAVERSE:" << endl;
cout << "-----"
" << endl;
    for (int i = 0; i < n; i++) {
        if (i % 2 == 0) {
            // baris genap: kiri ke kanan
            for (int j = 0; j < n; j++) {
                if (transpose[i][j] < 10) cout << " 0" <<
transpose[i][j];
                else cout << " " << transpose[i][j];
            }
        } else {
            // baris ganjil: kanan ke kiri
            for (int j = n - 1; j >= 0; j--) {
                if (transpose[i][j] < 10) cout << " 0" <<
transpose[i][j];
                else cout << " " << transpose[i][j];
            }
        }
        cout << endl;
    }
    cout << endl;
    cout << "-----"
" << endl;
    cout << endl << ">> [TASK 3/3] COMPLETE - Navigation matrix aligned."
<< endl;
    cout <<
"=====
==" << endl;
    cout << "  ALL TASK COMPLETE - SHIP SYSTEM RESTORED" << endl;
    cout <<
"=====
==" << endl;
    return 0;
}

```

5. Hasil Program

```
[ TASK 1/3 ] DECODE SIGNAL - Communication Room
>> Kirim sinyal terenkripsi dan deskripsi balasan dari Bumi!.
[ ENKRIPSI ]
Masukkan key           : 3
Masukkan teks          : Si Hitam Impostornya!

Matriks Enkripsi:
S i  H
i t a m
   I m p
o s t o
r n y a
! _ _ _

Enkripsi sebelum ditambah key : Si or!itIsn_ amty_Hmpoa_
Enkripsi setelah ditambah key  : Vl#ru$lwLvqb#dpw|bKpsrdb
```

Gambar 3.9 Tampilan Task 1: Enkripsi dengan Matriks 4 Kolom.

Berdasarkan **Gambar 3.9** di atas, dapat diketahui bahwa ketika pengguna memasukkan *key* bernilai "3" dan teks "Si Hitam Impostornya!", program akan menyusun teks tersebut ke dalam matriks dengan lebar tetap 4 kolom. Karakter yang kosong pada baris terakhir akan diisi dengan simbol *underscore* ("_"). Program kemudian melakukan enkripsi dengan membaca karakter per kolom dan menggeser nilai ASCII setiap karakter sejauh nilai *key* yang telah ditentukan.

```
[ DESKRIPSI ]
Masukkan teks terenkripsi      : Vl#ru$lwLvqb#dpw|bKpsrdb
Dekripsi setelah dikurang key  : Si or!itIsn_ amty_Hmpoa_

Matriks Dekripsi:
S i  H
i t a m
   I m p
o s t o
r n y a
! _ _ _

Hasil Dekripsi Final   : Si Hitam Impostornya!
>> [TASK 1/3] COMPLETE - Signal channel restored.
Tekan ENTER untuk melanjutkan ke [ TASK 2 ] !
Press any key to continue . . .
```

Gambar 3.10 Tampilan Task 1: Dekripsi.

Berdasarkan **Gambar 3.10** di atas, dapat diketahui bahwa ketika pengguna memasukkan teks terenkripsi, program akan melakukan proses dekripsi dengan mengurangi nilai ASCII setiap karakter sejauh nilai *key* yang telah ditentukan sebelumnya. Karakter hasil pengurangan tersebut kemudian disusun kembali ke dalam matriks dekripsi secara vertikal (per kolom). Program akhirnya menampilkan hasil dekripsi final dengan membaca matriks secara horizontal (per baris) dan mengabaikan simbol *underscore* ("_") sehingga pesan asli kembali terbaca dengan utuh.

```
[TASK 2/3] REPAIR TEXT CONSOLE - Captain's Bridge
>> Konsol teks mengalami interfensi kosmik.
>> Verifikasi integritas data teks sebelum terlambat!

[VERIFIKASI TEKS]

Masukkan Kalimat ke-1 : kasur rusak
-----
Hasil untuk Kalimat ke-1:
Jumlah Huruf : 10
1. Asli : k, a, s, u, r, r, u, s, a, k
2. Inverse : k, a, s, u, r, r, u, s, a, k
3. Swap Awal-Akhir : kasurruusak
4. Palindrome? : YES
```

Gambar 3.11 Tampilan *Task 2*: Pemrosesan Kalimat *Loop* Pertama.

Berdasarkan **Gambar 3.11** di atas, dapat diketahui bahwa ketika pengguna memasukkan kalimat pertama "kasur rusak", program akan menghitung jumlah huruf dan menampilkan urutan karakter secara asli serta terbalik (*inverse*). Program juga melakukan pertukaran (*swap*) antara karakter pertama dan terakhir dari kalimat tersebut. Terakhir, program melakukan pengecekan integritas untuk menentukan apakah kalimat tersebut merupakan sebuah *palindrome* atau bukan.

```
Masukkan Kalimat ke-2 : pipip
-----
Hasil untuk Kalimat ke-2:
Jumlah Huruf : 5
1. Asli : p, i, p, i, p
2. Inverse : p, i, p, i, p
3. Swap Awal-Akhir : pipip
4. Palindrome? : YES

Masukkan Kalimat ke-3 : kicau
-----
Hasil untuk Kalimat ke-3:
Jumlah Huruf : 5
1. Asli : k, i, c, a, u
2. Inverse : u, a, c, i, k
3. Swap Awal-Akhir : uicak
4. Palindrome? : NO

=====
Verifikasi Teks Lain? (y/n): n

>> [TASK 2/3] COMPLETE - Text console back online.
>> Tekan ENTER untuk melanjutkan ke [ TASK 3 ]!.
Press any key to continue . . .
```

Gambar 3.12 Tampilan *Task 2*: Pemrosesan Kalimat *Loop* Kedua dan Ketiga.

Berdasarkan **Gambar 3.12** di atas, dapat diketahui bahwa program melanjutkan proses verifikasi untuk kalimat kedua dan ketiga secara berurutan. Setelah semua data kalimat diproses, program akan memberikan opsi kepada pengguna untuk melakukan verifikasi teks lain. Jika pengguna memilih untuk tidak melanjutkan, maka tugas kedua dinyatakan selesai dan sistem akan meminta pengguna menekan tombol *enter* untuk menuju ke tahapan berikutnya.

```
[ TASK 3/3 ] ALIGN NAVIGATION MATRIX - Engine Core
>> Sistem navigasi mati total akibat sabotase.
>> Rekonstruksi matriks koordinat sekarang!

[ MATRIX NAVIGATION ]

Masukkan ukuran Navigation Matrix (min 3, max 5): 3
Odd Coordinate Matrix (Ganjil):
01 03 05
07 09 11
13 15 17

Even Coordinate Matrix (Genap):
02 04 06
08 10 12
14 16 18
-----

Matriks koordinat telah dipulihkan
Lakukanlah proses-proses berikut ini untuk membuat matriks koordinat lebih a
kurat!
1. Coordinate Matrix Integration
2. Matrix Transpose
3. Snake Traverse

Tekan ENTER untuk melanjutkan...
Press any key to continue . . .
```

Gambar 3.13 Tampilan *Task 3*: Pembuatan Matriks Ganjil dan Genap.

Berdasarkan **Gambar 3.13** di atas, dapat diketahui bahwa ketika pengguna memasukkan ukuran matriks koordinat bernilai "3", program secara otomatis menghasilkan dua matriks berbeda, yaitu matriks ganjil dan matriks genap. Matriks

ganjil diisi dengan deret bilangan ganjil mulai dari 1, sedangkan matriks genap diisi dengan deret bilangan genap mulai dari 2. Kedua matriks tersebut berfungsi sebagai data dasar yang akan diproses lebih lanjut untuk meningkatkan akurasi koordinat navigasi.

```

COORDINATE MATRIX INTEGRATION:
-----
03 07 11
15 19 23
27 31 35

MATRIX TRANSPOSE:
-----
03 15 27
07 19 31
11 23 35

```

Gambar 3.14 Tampilan Task 3: Pemrosesan Operasi *Matrix Fusion* dan *Transpose*.

Berdasarkan **Gambar 3.14** di atas, dapat diketahui bahwa program melakukan proses integrasi dengan menjumlahkan setiap elemen pada matriks ganjil dan genap yang berada di posisi yang sama. Setelah integrasi selesai, program menjalankan operasi *transpose* untuk mengubah susunan baris menjadi kolom pada matriks hasil integrasi tersebut. Proses ini merupakan bagian dari rekonstruksi sistem navigasi guna mendapatkan titik koordinat yang lebih akurat.

```

SNAKE TRAVERSE:
-----
03 15 27
31 19 07
11 23 35

-----
>> [TASK 3/3] COMPLETE - Navigation matrix aligned.
=====
ALL TASK COMPLETE - SHIP SYSTEM RESTORED
=====

Process exited after 639.9 seconds with return value 0
Press any key to continue . . .

```

Gambar 3.15 Tampilan Task 3: Snake Traverse dan Akhir Program.

Berdasarkan **Gambar 3.15** di atas, dapat diketahui bahwa program melakukan proses *Snake Traverse* pada matriks navigasi dengan cara menampilkan elemen baris genap dari arah kiri ke kanan dan elemen baris ganjil dari arah kanan ke kiri. Setelah seluruh proses navigasi selesai, sistem akan menampilkan pesan konfirmasi bahwa tugas ketiga telah lengkap dan seluruh sistem kapal telah berhasil dipulihkan. Tampilan akhir juga menunjukkan informasi mengenai waktu eksekusi program sebelum pengguna menekan sembarang tombol untuk menutup konsol

3.5 ANALISIS PROGRAM

```
#include <iostream>
```

Snippet “`#include <iostream>`” di atas merupakan *header file* standar dalam Bahasa C++ yang digunakan untuk menangani operasi masukan dan keluaran. *Header* ini memungkinkan penggunaan perintah seperti “`cin`” untuk *input* dan “`cout`” untuk *output* dalam program.

```
using namespace std;
```

Snippet “`using namespace std;`” di atas merupakan pernyataan untuk menggunakan seluruh fungsi dan objek dalam namespace `std`. Hal ini membuat penulisan kode lebih ringkas karena program tidak perlu lagi menambahkan awalan `std::` pada setiap elemen standar yang digunakan.

```
int main(){
```

Snippet “`int main(){`” di atas merupakan fungsi utama yang wajib dalam C++ di mana seluruh logika program dijalankan. Kata kunci `int` menunjukkan fungsi akan mengembalikan nilai *integer* ke sistem operasi, sementara tanda kurung kurawal “`{ }`” berfungsi sebagai pembuka dan penutupi blok instruksi program.

```
const int MAX = 100;
int key;
char teks [MAX];
cout << "[ TASK 1/3 ] DECODE SIGNAL - Communication Room" << endl << endl;
cout << ">> Kirim sinyal terenkripsi dan deskripsi balasan dari Bumi!."
<< endl << endl;
cout << "[ ENKRIPSI ]" << endl << endl;
do{
    cout << "Masukkan key \t\t: "; cin >> key;
    if (cin.fail() || key <= 0){
        cin.clear();
        cin.ignore (MAX, '\n');
        cout << "[!] Key harus bilangan bulat positif. Coba lagi" << endl <<
endl;
    }
}while (key <= 0);
```

Snippet di atas merupakan bagian yang menampilkan judul tugas pertama dengan tema antariksa. Bagian *do-while* berfungsi sebagai perintah validasi untuk memastikan pengguna memasukkan angka bulat positif sebagai kunci pergeseran. Jika *input* tidak valid (seperti huruf), perintah “`cin.clear()`” akan mereset status kesalahan dan “`cin.ignore()`” akan membersihkan sisa *input* agar program tidak mengalami gangguan atau *error*.

```

cin.ignore (MAX, '\n');
do{
    cout << "Masukkan teks \t\t: ";
    cin.getline (teks, MAX);
    cout << endl;
    if (teks == '\0'){
        cout << "[!] Input tidak boleh kosong. Coba lagi" << endl <<
endl;
    }
}while(teks == '\0');
int panjang = 0;
while (teks [panjang] != '\0'){
    panjang++;
}
int baris = (panjang + 3) / 4;

```

Snippet di atas merupakan perintah untuk mengambil input kalimat utuh dari pengguna. Karakter `\0` digunakan untuk mengecek apakah *input* kosong atau tidak. Baris kode “`int baris = (panjang + 3) / 4;`” merupakan perintah perhitungan aritmatika untuk menentukan jumlah baris yang dibutuhkan agar teks dapat tersusun rapi dalam format matriks statis dengan lebar tetap 4 kolom.

```

char matriks [baris][4];
int indeks = 0;
for (int i = 0 ; i < baris ; i++){
    for (int j = 0 ; j < 4 ; j++){
        if (indeks < panjang){
            matriks [i][j] = teks [indeks];
        }
        else {
            matriks [i][j] = '_';
        }
        indeks++;
    }
}

```

Snippet di atas merupakan perintah perulangan bersarang (*nested loop*) yang berfungsi untuk menyusun karakter teks ke dalam matriks secara horizontal atau baris demi baris. Perintah matriks “`matriks [i][j] = '_';`” berfungsi sebagai instruksi untuk mengisi sel matriks yang kosong dengan simbol *underscore* sebagai *padding* agar struktur matriks tetap penuh dan simetris.


```
cout << "Matriks Enkripsi: " << endl << endl;

for (int i = 0 ; i < baris ; i++){
    for (int j = 0 ; j < 4 ; j++){
        cout << matriks [i][j] << " ";
    }
    cout << endl << endl;
}
```

Snippet di atas merupakan struktur yang digunakan untuk menampilkan struktur matriks yang telah terbentuk ke layar agar pengguna dapat melihat posisi setiap karakter. Perulangan luar (i) mencetak baris baru, sementara perulangan dalam (j) mencetak karakter setiap kolom dalam satu baris diikuti spasi sebagai pemisah visual.

```
cout << "Enkripsi sebelum ditambah key \t: ";
for (int j = 0; j < 4; j++) {
    for (int i = 0; i < baris; i++) {
        cout << matriks[i][j];
    }
}
cout << endl;
cout << "Enkripsi setelah ditambah key \t: ";

for (int j = 0; j < 4; j++) {
    for (int i = 0; i < baris; i++) {
        cout << (char)(matriks[i][j] + key);
    }
}
cout << endl << endl;
```

Snippet di atas merupakan inti dari proses. Program menerapkan metode pembacaan matriks secara vertikal (*column-major order*), di mana perulangan luar mengontrol indeks kolom (j) dan perulangan dalam mengontrol indeks baris (i). Teknik ini secara efektif mengacak urutan karakter asli yang sebelumnya disusun secara horizontal (baris demi baris).

Pada blok kedua, program menjalankan instruksi transformasi data yang lebih kompleks menggunakan operasi aritmatika pada nilai ASCII. Perintah “(char)(matriks[i][j] + key);” berfungsi sebagai instruksi untuk menggeser posisi karakter asli sejauh nilai kunci (*key*) yang telah divalidasi sebelumnya. Hasil dari pergeseran ini dikonversi kembali ke tipe data karakter (*type casting*), sehingga menghasilkan rangkaian kode rahasia yang terenkripsi sepenuhnya dan siap untuk ditransmisikan melalui kanal komunikasi antariksa.

```

c cout << "[ DESKRIPSI ]" << endl << endl;
  char enkripsi[MAX];

  do {
    cout << "Masukkan teks terenkripsi \t: ";
    cin.getline(enkripsi, MAX);

    if (enkripsi == '\0') {
      cout << "[!] Input tidak boleh kosong. Coba lagi" << endl;
    }
  } while (enkripsi == '\0');

  int panjangEnkripsi = 0;
  while (enkripsi[panjangEnkripsi] != '\0') {
    panjangEnkripsi++;
  }

```

Snippet di atas merupakan instruksi untuk menerima teks yang akan didekripsi. Validasi *do-while* digunakan agar pengguna tidak memasukkan data kosong. Setelah input diterima, program menjalankan perintah perulangan *while* untuk menghitung jumlah karakter dalam teks terenkripsi tersebut sebagai persiapan tahap pembalikan nilai.

```

for (int i = 0; i < panjangEnkripsi; i++) {
  enkripsi[i] = enkripsi[i] - key;
}
cout << "Dekripsi setelah dikurang key \t: ";
for (int i = 0; i < panjangEnkripsi; i++) {
  cout << enkripsi[i];
}
cout << endl << endl;
int barisDekripsi = (panjangEnkripsi + 3) / 4;
char matriksDekripsi[MAX];
int indeks2 = 0;
for (int j = 0; j < 4; j++) {
  for (int i = 0; i < barisDekripsi; i++) {
    if (indeks2 < panjangEnkripsi) {
      matriksDekripsi[i][j] = enkripsi[indeks2];
    } else {
      matriksDekripsi[i][j] = '_';
    }
    indeks2++;
  }
}

```

Snippet di atas merupakan bagian menjalankan perintah pengurangan nilai ASCII karakter dengan *key* untuk mengembalikan teks ke bentuk aslinya. Data tersebut dimasukkan kembali ke dalam “*matriksDekripsi*” dengan pola pembacaan kolom agar pesan yang tadinya teracak secara vertikal dapat disusun kembali pada posisi baris yang benar.

```

cout << "Matriks Dekripsi:" << endl << endl;
for (int i = 0; i < barisDekripsi; i++) {
    for (int j = 0; j < 4; j++) {
        cout << matriksDekripsi[i][j] << " ";
    }
    cout << endl;
}

cout << endl << "Hasil Dekripsi Final \t: ";

for (int i = 0; i < barisDekripsi; i++) {
    for (int j = 0; j < 4; j++) {
        if (matriksDekripsi[i][j] != '_') {
            cout << matriksDekripsi[i][j];
        }
    }
}

```

Snippet di atas merupakan instruksi untuk menyajikan data hasil pemulihan sinyal kepada pengguna. Blok perulangan pertama berfungsi untuk menampilkan struktur matriks dekripsi secara visual guna memverifikasi susunan karakter setelah proses pembalikan nilai ASCII. Program membaca matriks secara horizontal (baris demi baris) untuk memulihkan alur teks asli.

Selanjutnya, pada bagian Hasil Dekripsi Final, program menerapkan logika penyaringan data yang krusial. Perintah kondisional “if (matriksDekripsi[i][j] != '_')” bertindak sebagai filter cerdas untuk mengabaikan karakter *underscore* () yang sebelumnya digunakan sebagai *padding* atau pengisi sel kosong. Hasilnya, program dapat merekonstruksi dan menampilkan pesan komunikasi asli secara bersih, utuh, dan profesional tanpa adanya simbol teknis yang mengganggu keterbacaan teks.

```

cout << ">> [TASK 1/3] COMPLETE - Signal channel restored." << endl;
cout << "Tekan ENTER untuk melanjutkan ke [ TASK 2 ] !" << endl;
system("pause");
system("cls");
cout << endl;

```

Snippet di atas merupakan perintah transisi antar tugas. “system(“pause”)” menghentikan sementara jalannya program hingga pengguna menekan tombol tertentu, sementara “system(“cls”)” berfungsi sebagai instruksi untuk membersihkan seluruh tampilan di jendela konsol agar persiapan tugas berikutnya terlihat lebih rapi.

```

cout << "[TASK 2/3] REPAIR TEXT CONSOLE - Captain's Bridge" << endl;
    cout << ">> Konsol teks mengalami interfensi kosmik." << endl;
    cout << ">> Verifikasi integritas data teks sebelum terlambat!" << endl
<< endl;

    char pilih;
    do {
        cout << "[VERIFIKASI TEKS]" << endl << endl;
        char kalimat1[MAX];
        cin.ignore(MAX, '\n');
        cout << "Masukkan Kalimat ke-1 \t: ";
        cin.getline(kalimat1, MAX);
        int panjangKalimat1 = 0;
        int jumlah1 = 0;
        while (kalimat1[panjangKalimat1] != '\0') {
            if (kalimat1[panjangKalimat1] != ' ') {
                jumlah1++;
            }
            panjangKalimat1++;
        }
    }

```

Snippet di atas merupakan pembuka untuk tugas kedua yang berfungsi menyajikan antarmuka pengguna berbasis narasi antariksa. Program memanfaatkan struktur perulangan *do-while* sebagai kerangka utama yang memungkinkan pengguna melakukan proses verifikasi teks secara berulang kali sesuai kebutuhan tanpa harus merestart program dari awal.

Di dalam blok perulangan tersebut, terdapat perintah “`cin.ignore(MAX, '\n')`” yang memegang peranan krusial sebagai instruksi pembersihan *buffer*. Perintah ini memastikan tidak ada sisa karakter (seperti karakter *newline* dari *input* sebelumnya) yang tertinggal di memori, sehingga fungsi “`cin.getline(kalimat1, MAX)`” dapat bekerja secara optimal untuk menangkap seluruh rangkaian kata dalam satu baris, termasuk spasi, hingga batas maksimal yang ditentukan oleh konstanta “MAX”.

Selanjutnya, program menjalankan algoritma analisis teks menggunakan *while* untuk menelusuri *array* “`kalimat1`” karakter demi karakter. Proses navigasi ini berlangsung secara kontinu hingga program mendeteksi *null terminator* (`\0`) yang menandakan akhir dari *string*. Di dalam perulangan ini, diterapkan logika kondisional “`if (kalimat1[panjangKalimat1] != ' ')`” yang secara spesifik berfungsi untuk menghitung “`jumlah1`” dengan mengeliminasi karakter spasi, sementara variabel “`panjangKalimat1`” secara simultan menghitung total panjang keseluruhan *string* tersebut. Proses ini merupakan tahap dasar yang sangat penting sebelum melakukan verifikasi integritas data lebih lanjut.

```

cout << "-----" << endl;
cout << "Hasil untuk Kalimat ke-1:" << endl;
cout << "Jumlah Huruf \t\t: " << jumlah1 << endl;
cout << "1. Asli \t\t: ";
for (int i = 0; i < panjangKalimat1; i++) {
    if (kalimat1[i] != ' ') {
        cout << kalimat1[i];
        if (i < panjangKalimat1 - 1) {
            cout << ", ";
        }
    }
}
cout << endl;
cout << "2. Inverse \t\t: ";
for (int i = panjangKalimat1 - 1; i >= 0; i--) {
    if (kalimat1[i] != ' ') {
        cout << kalimat1[i];
        if (i > 0) {
            cout << ", ";
        }
    }
}
cout << endl;
// cari index pertama dan terakhir yang bukan spasi
int awal = 0;
int akhir = panjangKalimat1 - 1;
while (kalimat1[awal] == ' ') awal++;
while (kalimat1[akhir] == ' ') akhir--;

```

Snippet di atas merupakan penyajian hasil pemrosesan teks kepada pengguna. Diawali dengan pencetakan garis pemisah dan informasi “jumlah1”, program kemudian menggunakan iterasi terarah melalui perulangan *for*. Blok pertama menampilkan karakter dalam urutan normal (Asli), sedangkan blok kedua menggunakan indeks menurun untuk menampilkan karakter dari belakang ke depan (Inverse). Di dalam kedua proses ini, terdapat filter “if (kalimat1[i] != ' ')” untuk mengabaikan spasi dan menyisipkan tanda koma sebagai pemisah visual.

Di bagian akhir, program menjalankan instruksi pencarian batas karakter menggunakan dua perulangan *while*. Prosedur ini mengidentifikasi indeks pertama (awal) dan terakhir (akhir) yang bukan merupakan spasi. Penentuan titik koordinat ini sangat penting sebagai tahap persiapan teknis sebelum melakukan operasi pertukaran karakter (*swap*) pada proses verifikasi selanjutnya.

```

char simpan = kalimat1[awal];
kalimat1[awal] = kalimat1[akhir];
kalimat1[akhir] = simpan;

cout << "3. Swap Awal-Akhir \t: ";
for (int i = 0; i < panjangKalimat1; i++) {
    if (kalimat1[i] != ' ') {
        cout << kalimat1[i];
    }
}
cout << endl;

int kiri = 0;
int kanan = panjangKalimat1 - 1;
bool palindrome = (jumlah1 != 0);

while (kiri < kanan) {
    while (kalimat1[kiri] == ' ') kiri++;
    while (kalimat1[kanan] == ' ') kanan--;
    if (kalimat1[kiri] != kalimat1[kanan]) {
        palindrome = false;
        break;
    }
    kiri++;
    kanan--;
}
cout << "4. Palindrome? \t\t: ";
if (palindrome) {
    cout << "YES" << endl;
} else {
    cout << "NO" << endl;
}
cout << endl;

```

Snippet di atas merupakan instruksi pertukaran (*swap*) karakter pertama dan terakhir (non-spasi) menggunakan variabel bantuan “simpan”. Hasil pertukaran tersebut ditampilkan kembali ke layar menggunakan perulangan *for* yang memfilter karakter spasi.

Selanjutnya, program melakukan verifikasi *palindrome* untuk mengecek integritas teks dengan membandingkan karakter dari kedua ujung (kiri dan kanan) secara simetris. Di dalam perulangan *while*, terdapat instruksi internal untuk mengabaikan spasi agar pengecekan tetap akurat. Jika ditemukan ketidakcocokan, status berubah menjadi *false* dan proses dihentikan dengan perintah *break*. Hasil akhir ditampilkan sebagai "YES" atau "NO" untuk menunjukkan validitas data. Algoritma yang sama berlaku identik untuk kalimat ke-2 dan ke-3 guna memastikan standar verifikasi yang seragam.

```

do {
    cout << "Verifikasi Teks Lain? (y/n): ";
    cin >> pilih;
    if (cin.fail() || (pilih != 'y' && pilih != 'Y' && pilih != 'n' &&
    pilih != 'N')) {
        cin.clear();
        cin.ignore(MAX, '\n');
        cout << "[!] Input tidak valid. Masukkan y atau n." << endl;
    }
    } while (cin.fail() || (pilih != 'y' && pilih != 'Y' && pilih != 'n' &&
    pilih != 'N'));

    } while (pilih == 'y' || pilih == 'Y');

    cout << endl << ">> [TASK 2/3] COMPLETE - Text console back online." <<
    endl;
    cout << ">> Tekan ENTER untuk melanjutkan ke [ TASK 3 ]!." << endl;
    system("pause");
    system("cls");
    cout << endl;

```

Snippet di atas merupakan instruksi kontrol melalui perulangan bersarang (*nested loop*). Program menggunakan *do-while* internal untuk memvalidasi *input* pengguna agar hanya menerima karakter 'y' atau 'n', di mana perintah “*cin.clear()*” dan “*cin.ignore()*” berfungsi mereset sistem jika terjadi kesalahan *input*. Perulangan luar bertugas mengulang seluruh rangkaian verifikasi teks selama variabel pilih bernilai 'y'. Setelah selesai, program menjalankan instruksi “*system("cls")*” untuk membersihkan jendela konsol sebelum bertransisi ke tahap berikutnya

```

cout << "[ TASK 3/3 ] ALIGN NAVIGATION MATRIX - Engine Core" << endl;
cout << ">> Sistem navigasi mati total akibat sabotase." << endl;
cout << ">> Rekonstruksi matriks koordinat sekarang!" << endl << endl;
cout << "[ MATRIX NAVIGATION ]" << endl << endl;
int n;
do {
    cout << "Masukkan ukuran Navigation Matrix (min 3, max 5): ";
    cin >> n;
    if (cin.fail() || n < 3 || n > 5) {
        cin.clear();
        cin.ignore(MAX, '\n');
        cout << "[!] Input tidak valid. Masukkan ukuran antara 3 sampai
    5." << endl << endl;
    }
    } while (cin.fail() || n < 3 || n > 5);

```

Snippet di atas merupakan pembuka untuk tugas ketiga, berfungsi sebagai *header* program. Program menggunakan *do-while* untuk meminta pengguna memasukkan matriks dengan batasan minimal 3 dan maksimal 5. Terdapat validasi “*if (cin.fail() || n < 3 || n > 5)*” untuk memastikan bahwa *input* yang dimasukkan adalah angka dalam rentang yang benar, jika tidak valid, perintah “*cin.clear()*” dan *cin.ignore()*” akan mereset status *error* sistem sebelum meminta pengguna melakukan *input* ulang.

```

int ganjil[5][5];
int genap[5][5];
cout << "Odd Coordinate Matrix (Ganjil):" << endl;
int nilaiGanjil = 1;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ganjil[i][j] = nilaiGanjil;
        if (ganjil[i][j] < 10) cout << " 0" << ganjil[i][j];
        else cout << " " << ganjil[i][j];
        nilaiGanjil += 2;
    }
    cout << endl;
}
cout << endl;
cout << "Even Coordinate Matrix (Genap):" << endl;
int nilaiGenap = 2;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        genap[i][j] = nilaiGenap;
        if (genap[i][j] < 10) cout << " 0" << genap[i][j];
        else cout << " " << genap[i][j];
        nilaiGenap += 2;
    }
    cout << endl;
}
cout << "-----" <<
endl;
cout << endl << "Matriks koordinat telah dipulihkan" << endl;
cout << "Lakukanlah proses-proses berikut ini untuk membuat matriks
koordinat lebih akurat!" << endl;
cout << "1. Coordinate Matrix Integration" << endl;
cout << "2. Matrix Transpose" << endl;
cout << "3. Snake Traverse" << endl << endl;
cout << "Tekan ENTER untuk melanjutkan..." << endl;
system("pause");
system("cls");

```

Snippet di atas merupakan instruksi untuk menginisialisasi dua buah matriks koordinat, yaitu “ganjil” dan “genap”. Program menggunakan perulangan bersarang (*nested loop*) untuk mengisi matriks tersebut dengan deret bilangan ganjil (dimulai dari 1) dan bilangan genap (dimulai dari 2) secara berurutan. Di dalam setiap perulangan, terdapat perintah kondisional “if (ganjil[i][j] < 10)” dan “if (genap[i][j] < 10)” yang berfungsi sebagai instruksi pemformatan visual untuk menambahkan karakter "0" di depan angka satuan, sehingga tampilan matriks koordinat pada layar tetap sejajar dan rapi. Setelah data dipulihkan, program menampilkan daftar proses lanjutan yang harus dilakukan dan menjalankan perintah “system("cls")” untuk membersihkan layar sebelum masuk ke tahap integrasi.


```

int gabungGG[5][5];
cout << "COORDINATE MATRIX INTEGRATION:" << endl;
cout << "-----" <<
endl;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        gabungGG[i][j] = ganjil[i][j] + genap[i][j];
        if (gabungGG[i][j] < 10) cout << " 0" << gabungGG[i][j];
        else cout << " " << gabungGG[i][j];
    }
    cout << endl;
}
cout << endl;

```

Snippet di atas merupakan instruksi untuk melakukan proses integrasi data navigasi melalui penggabungan dua matriks. Program menggunakan perulangan bersarang (*nested loop*) untuk menjumlahkan setiap elemen dari matriks “ganjil” dan “genap” yang berada pada posisi indeks yang sama, lalu menyimpannya ke dalam matriks baru bernama “gabungGG”. Di dalam perulangan tersebut, terdapat perintah kondisional “if (gabungGG[i][j] < 10)” yang berfungsi sebagai instruksi pemformatan untuk mencetak angka nol di depan bilangan satuan agar tampilan seluruh baris dan kolom pada koordinat integrasi tetap sejajar dan simetris.

```

cout << "MATRIX TRANSPOSE:" << endl;
cout << "-----" <<
endl;
int transpose[5][5];
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        transpose[i][j] = gabungGG[j][i];
        if (transpose[i][j] < 10) cout << " 0" << transpose[i][j];
        else cout << " " << transpose[i][j];
    }
    cout << endl;
}
cout << endl;

```

Snippet di atas merupakan instruksi untuk mengubah struktur matriks hasil integrasi melalui operasi *transpose*. Program menggunakan perulangan bersarang (*nested loop*) untuk menukar posisi baris menjadi kolom dengan cara memindahkan elemen dari “gabungGG[j][i]” ke dalam “transpose[i][j]”. Di dalam perulangan tersebut, terdapat perintah kondisional “if (transpose[i][j] < 10)” yang berfungsi sebagai instruksi pemformatan visual untuk mencetak angka nol di depan bilangan satuan agar susunan matriks koordinat yang baru tetap terlihat sejajar dan terorganisir di layar.

```

cout << "SNAKE TRAVERSE:" << endl;
cout << "-----" <<
endl;
for (int i = 0; i < n; i++) {
    if (i % 2 == 0) {
        // baris genap: kiri ke kanan
        for (int j = 0; j < n; j++) {
            if (transpose[i][j] < 10) cout << " 0" << transpose[i][j];
            else cout << " " << transpose[i][j];
        }
    } else {
        // baris ganjil: kanan ke kiri
        for (int j = n - 1; j >= 0; j--) {
            if (transpose[i][j] < 10) cout << " 0" << transpose[i][j];
            else cout << " " << transpose[i][j];
        }
    }
    cout << endl;
}

```

Snippet di atas merupakan instruksi final untuk menampilkan data navigasi menggunakan pola *Snake Traverse*. Program menggunakan perulangan bersarang dengan perintah kondisional “if (i % 2 == 0)” untuk menentukan arah pencetakan baris. Baris indeks genap dicetak secara normal dari kiri ke kanan, sedangkan bagian *else* (baris indeks ganjil) menjalankan instruksi perulangan mundur dari kanan ke kiri. Sama seperti bagian sebelumnya, terdapat perintah pemformatan untuk mencetak angka nol di depan bilangan satuan agar seluruh koordinat tetap sejajar. Proses ini menghasilkan jalur berkelok yang merepresentasikan rute akhir sistem navigasi kapal.

```

cout << endl;
cout << "-----" <<
endl;
cout << endl << ">> [TASK 3/3] COMPLETE - Navigation matrix aligned." <<
endl;
cout <<
"=====
<< endl;
    cout << "  ALL TASK COMPLETE - SHIP SYSTEM RESTORED" << endl;
    cout <<
"=====
<< endl;
    return 0;
}

```

Snippet di atas merupakan instruksi final untuk menampilkan pesan penyelesaian seluruh tugas ke jendela konsol. Program menampilkan baris penutup yang menegaskan bahwa seluruh sistem kapal telah berhasil dipulihkan secara total.

```
return 0;
```

Snippet “return 0;” di atas merupakan perintah yang digunakan untuk mengakhiri fungsi “main()” dengan mengembalikan nilai 0 kepada sistem operasi. Nilai 0 menandakan bahwa program telah berjalan dengan sukses tanpa adanya kesalahan.

3.6 KESIMPULAN

Berdasarkan hasil praktikum pada Modul III tentang *Array*, dapat diambil kesimpulan bahwa:

1. *Array* adalah struktur data yang digunakan untuk menyimpan sekumpulan nilai dengan tipe data yang sama dalam satu variabel yang diakses menggunakan indeks. *Array* dapat berdimensi satu untuk menyimpan deretan data linear, maupun berdimensi dua untuk merepresentasikan data dalam bentuk matriks baris dan kolom. Pemahaman terhadap konsep indeks, ukuran, dan batas *array* sangat penting agar program dapat mengakses dan mengelola data secara tepat dan efisien.
2. Program yang menggunakan *array* dapat dibangun untuk menyelesaikan berbagai permasalahan seperti penyimpanan data, pengurutan, pencarian, hingga operasi matriks. Pengisian dan pembacaan elemen *array* dilakukan melalui perulangan yang mengakses setiap indeks secara sistematis, baik per baris maupun per kolom, sehingga data dapat diproses sesuai kebutuhan program yang dirancang.
3. Manipulasi *string* dalam C++ pada dasarnya merupakan manipulasi *array* bertipe “char” yang diakhiri karakter null ('\0'). Operasi-operasi pada *string* seperti menghitung panjang, membalik urutan karakter, menukar karakter pada posisi tertentu, menyalin isi *string*, serta memeriksa kesamaan dua *string* dapat dilakukan secara manual dengan mengakses setiap elemen karakter melalui indeks tanpa bergantung pada *library* tambahan.
4. Program yang mampu memanipulasi *string* dapat diimplementasikan dengan memanfaatkan perulangan dan logika pengkondisian untuk mengolah setiap karakter dalam *array*. Operasi seperti *inverse*, *swap* karakter awal dan akhir, pengecekan *palindrome*, enkripsi dengan pergeseran nilai ASCII, serta pengisian dan pembacaan matriks karakter merupakan contoh nyata bagaimana manipulasi *string* berbasis *array* diterapkan dalam sebuah program yang fungsional.

DAFTAR PUSTAKA

- [1] R. S. Pressman dan B. Maxim, *Rekayasa Perangkat Lunak: Pendekatan Praktisi*, Yogyakarta: Andi, 2015.
- [2] B. Nugroho, *Dasar Pemrograman C++*, Yogyakarta: Gava Media, 2017.
- [3] E. Winarno dan Z. A. Hasibuan, *Pemrograman C++ untuk Pemula*, Jakarta: Elex Media Komputindo, 2016.
- [4] A. Kadir, *Algoritma dan Pemrograman Menggunakan C & C++*, Yogyakarta: Andi, 2018.